

Aula 1 (Módulo 07) — Repositórios, Commits e Branches

Você já vem usando Git desde a primeira aula deste repositório (`git add`, `git commit`, `git push`) — nesta aula formalizamos **o que** está acontecendo por trás desses comandos.

1. O que é Git (visão simples)

Git é um sistema de **controle de versão**: ele guarda o histórico completo de todas as mudanças feitas nos arquivos de um projeto ao longo do tempo — quem mudou o quê, quando, e por quê (se a mensagem explicar). Diferente de simplesmente ter várias cópias de um arquivo (`aula1.md`, `aula1_final.md`, `aula1_final_v2.md` ...), o Git guarda o histórico de forma estruturada e permite voltar a qualquer ponto anterior.

GitHub é um **serviço online** que hospeda repositórios Git na nuvem, além de adicionar funcionalidades de colaboração (Pull Requests, Issues — Aula 2). Git é a ferramenta; GitHub é uma plataforma que usa Git. Existem outras plataformas parecidas (GitLab, Bitbucket) — os conceitos de Git em si são os mesmos em qualquer uma delas.

2. Repositório

Um **repositório** (ou "repo") é uma pasta cujo histórico de mudanças o Git está rastreando. Dentro dela existe uma subpasta oculta `.git/` — é ali que **todo** o histórico fica guardado (nunca edite essa pasta manualmente).

```
git init          # transforma a pasta atual em um repositório novo
git clone <url>   # copia um repositório existente (como fizemos com este)
git status        # mostra o que mudou desde o último commit
```

Você já rodou `git status` várias vezes neste repositório — ele sempre te diz três coisas: arquivos **modificados**, arquivos **novos** (untracked) e o que já está **staged** (próxima seção).

3. A área de staging — o "carrinho de compras" do Git

Esse é o conceito que mais confunde iniciantes: entre "editar um arquivo" e "criar um commit", existe uma etapa intermediária chamada **staging area** (ou *index*).

```
Arquivo modificado → git add → Staging area → git commit → Histórico (commit)
```

```
git add arquivo.md    # adiciona UM arquivo específico ao staging
git add .              # adiciona TODOS os arquivos modificados/novos
git status             # confirma o que está staged antes de commitar
```

Por que essa etapa existe: ela permite montar um commit **só com o que faz sentido juntar** — por exemplo, se você editou dois arquivos sobre assuntos completamente diferentes, pode fazer `git add` só de um deles e criar dois commits separados, cada um com sua própria mensagem explicando exatamente aquela mudança. Isso é o que fizemos, aula

após aula, neste repositório: cada commit representa uma unidade de trabalho coerente (ex.: "Add Aula 2: sistemas operacionais...").

4. Commit — um "ponto de salvamento" com explicação

```
git commit -m "Adiciona aula sobre permissões de arquivo"
```

Um commit é uma **fotografia** do estado dos arquivos staged naquele momento, junto com: autor, data/hora, e uma **mensagem** explicando a mudança. Boas mensagens de commit são um hábito profissional — imagine voltar a um projeto depois de 6 meses e só ver mensagens como "fix" ou "mudanças": você não teria ideia do que foi feito sem abrir cada commit.

```
git log                # histórico completo de commits
git log --oneline      # versão resumida, uma linha por commit
```

5. Branch — trabalhar em paralelo sem bagunçar o principal

Uma **branch** ("ramo") é uma linha de desenvolvimento independente. A branch padrão geralmente se chama `main` (ou, historicamente, `master`). Criar uma nova branch permite fazer mudanças **sem afetar** a `main` até que você decida trazer (merge) essas mudanças de volta.

```
git branch              # lista as branches existentes
git branch nome-da-branch # cria uma branch nova
git checkout nome-da-branch # muda para essa branch
git checkout -b nome-da-branch # cria E já muda para ela (atalho)
```

Por que importa: branches permitem experimentar, corrigir um bug específico, ou desenvolver uma funcionalidade nova, **isoladamente** — se algo der errado, você simplesmente descarta a branch sem ter afetado o código principal. É exatamente por isso que todo o trabalho neste repositório está acontecendo numa branch própria (não diretamente na `main`) — um padrão profissional chamado **feature branch workflow**.

6. Merge — trazendo o trabalho de volta

```
git checkout main
git merge nome-da-branch
```

Isso pega todos os commits feitos na branch e os incorpora na `main`. Quando as mudanças não se sobrepõem a nada que mudou na `main` enquanto isso, o merge acontece automaticamente. Quando **duas branches mudaram a mesma linha do mesmo arquivo de formas diferentes**, acontece um **conflito de merge** — o Git marca exatamente onde está o conflito no arquivo, e cabe a você decidir qual versão (ou uma combinação) manter.

7. Remoto (`origin`) — sincronizando com o GitHub

```
git remote -v          # mostra os repositórios remotos configurados
git push origin main    # envia seus commits locais para o GitHub
git pull origin main    # traz commits novos do GitHub para o seu local
```

`origin` é só um **apelido** (por convenção) para a URL do repositório remoto — é o que permite `git push` / `git pull` sem digitar a URL inteira toda vez.

8. Conectando com o que já fizemos neste repositório

```
git status → o que mudou desde o último commit
git add    → move para staging (o que vai entrar no próximo commit)
git commit → salva um ponto no histórico, com mensagem explicando
git push   → envia para o GitHub (o remoto "origin")
```

Cada vez que uma nova aula foi adicionada aqui, esse ciclo se repetiu — agora você sabe exatamente o que cada passo faz.

9. Exercício prático

No seu computador, dentro de uma pasta de teste (não precisa ser este repositório):

```
mkdir teste-git && cd teste-git
git init
echo "primeira linha" > arquivo.txt
git add arquivo.txt
git commit -m "Primeiro commit"
git checkout -b minha-branch
echo "segunda linha" >> arquivo.txt
git add arquivo.txt
git commit -m "Adiciona segunda linha na branch de teste"
git log --oneline
git checkout main
cat arquivo.txt    # observe: a segunda linha NÃO aparece aqui, só na branch
git merge minha-branch
cat arquivo.txt    # agora aparece, depois do merge
```

10. Recapitulando

- Repositório = pasta com histórico rastreado pelo Git; GitHub = serviço que hospeda repositórios na nuvem.

- `git add` move para staging; `git commit` salva um ponto no histórico com mensagem; `git push` / `git pull` sincronizam com o remoto.
- Branch = linha de desenvolvimento isolada; merge = trazer de volta; conflito = quando a mesma linha muda de formas diferentes em duas branches.

Próxima aula

Pull Requests e Issues — como o GitHub formaliza revisão de código e rastreamento de tarefas/bugs, e por que isso importa mesmo para projetos pessoais de portfólio.

Fontes recomendadas

- **Git — documentação oficial ("Git Book")**: referência completa e gratuita, mantida pelo próprio projeto Git. <https://git-scm.com/book/en/v2>
- **GitHub Docs — "About Git"**: introdução oficial do GitHub aos conceitos de Git. <https://docs.github.com/get-started/using-git/about-git>